Plan de tests

Olympic Participation Tracker (Angular) et Workshop Organizer (Spring Boot) — version 1.0 du 15/10/2026 — Bruno Hoarau

Sommaire

Sommaire **1**
1. Introduction 2
2. Tests unitaires 3
    2.1 Périmètre et objectifs 3
    2.2 Outils et technologies 4
    2.3 Critères de réussite 4
    2.4 Commandes d'exécution et rapports 5
    2.5 Fréquence d'exécution 5
3. Tests d'intégration 7
    3.1 Périmètre et objectifs 7
    3.2 Outils et technologies recommandés 7
    3.3 Scénarios de test prioritaires 8
    3.4 Critères de réussite 9
    3.5 Fréquence d'exécution recommandée 9
4. Tests de performance 10
    4.1 Périmètre et objectifs 10
    4.2 Outils et technologies recommandés 10
    4.3 Métriques et seuils recommandés 10
    4.4 Scénarios de charge 11
    4.5 Fréquence d'exécution recommandée 12
5. Tests de sécurité 13
    5.1 Périmètre et objectifs 13
    5.2 Outils et technologies recommandés 13
    5.3 Vulnérabilités à vérifier en priorité 14
    5.4 Critères de réussite recommandés 14
    5.5 Fréquence d'exécution recommandée 15
6. Fonctionnalités à tester 16
7. Fonctionnalités à ne pas tester 17
    8. Matrice récapitulative des tests 18
    Notes et références 20

1. Introduction

Ce plan de tests définit la stratégie de validation des deux applications du projet. Elles sont réunies dans un monorepo GitHub et testées, construites puis publiées par un pipeline GitHub Actions unique (`.github/workflows/ci.yml`). Objectif : garantir la qualité, la fiabilité, la performance et la sécurité des applications avant et après leur mise en production, en automatisant au maximum les contrôles dans la chaîne CI/CD.

Applications concernées

Application	Rôle fonctionnel	Architecture technique	Exécution
Olympic Participation Tracker `olympic-games-starter` v0.1.3	Tableau de bord de la participation des pays aux Jeux olympiques : ville et année des JO, nombre d'athlètes et de médailles.	Angular 20.3 (composants standalone, signaux, routeur avec resolver), TypeScript 5.8, RxJS 7.5. Données chargées par HttpClient depuis un fichier JSON statique (`assets/mock/olympic.json`) : aucune API distante.	Image Docker multi-étapes : build Node 22, service par Nginx 1.27 (port 80).
Workshop Organizer `workshop-organizer` v0.2.4	API REST de gestion d'ateliers (workshops) et des notions qu'ils abordent.	Java 21, Spring Boot 3.2.4 (Web, Data JPA, Validation), Gradle 8.7. Interfaces des contrôleurs générées depuis le contrat `openapi.yaml` (OpenAPI Generator 7.4), mappage DTO / entités par ModelMapper, persistance PostgreSQL avec une relation n-n ateliers / notions.	Image Docker : build Gradle, exécution eclipse-temurin 21 JRE (port 8080), base PostgreSQL 13.

Endpoints de l'API (préfixe `/api`) : `GET` et `POST /notions`, `GET` et `POST /workshops` (paramètre `limit`), `GET /workshops/{workshopId}`.

Les deux applications sont **indépendantes** : le front Olympic ne consomme pas l'API Workshop. Chaque application est donc traitée séparément, sans test de bout en bout croisé.

Démarche

- **Pyramide des tests** : beaucoup de tests unitaires rapides, moins de tests d'intégration, quelques tests système ciblés sur les parcours critiques ; performance et sécurité selon leur coût.
- **Shift-left** : les contrôles rapides (tests unitaires, scan de secrets) bloquent chaque push ; les plus coûteux (charge, scan dynamique) s'exécutent sur un environnement de pré-production.
- **Critères mesurables** : chaque type de test a des seuils chiffrés qui conditionnent la publication des images Docker et la mise en production.

État actuel : seuls des tests unitaires existent dans les applications. Le pipeline ajoute déjà deux contrôles automatisés : le scan de secrets (Gitleaks) et un test de fumée de chaque image Docker avant sa publication sur GHCR. Les tests d'intégration, de performance et de sécurité applicative décrits ici sont des recommandations pour une mise en production réelle ; la matrice (section 8) distingue ce qui est implémenté de ce qui est recommandé.

Rôles

Dev : écriture et maintenance des tests. **QA** : conception des scénarios et campagnes de recette. **Ops** : environnements, pipeline, performance. **DevSecOps** : outillage sécurité, tri et suivi des vulnérabilités. Sur ce projet, une même personne porte plusieurs rôles ; la répartition indique la responsabilité attendue dans une équipe.

2. Tests unitaires

Les tests unitaires vérifient chaque composant isolément, ses dépendances étant simulées. Ce sont les seuls tests présents dans le code ; ils s'exécutent à chaque push dans le job `test` du pipeline. Les mesures ci-dessous ont été réalisées le 15/10/2026 sur le commit `28b9346`.

2.1 Périmètre et objectifs

Olympic Participation Tracker : 4 fichiers, 5 tests, tous réussis

Fichier de test	Ce qui est vérifié	Ce qui n'est pas vérifié
`app.component.spec.ts`	Création du composant racine ; valeur du titre	Routage
`home.component.spec.ts`	Création du composant avec une liste vide	Affichage des pays et des participations
`not-found.component.spec.ts`	Création de la page 404	Contenu de la page
`olympic.service.spec.ts`	Instanciation du service	Appel de `getOlympics()`, gestion d'erreur

Couverture mesurée (`ng test --code-coverage`) : **75 % des instructions (9/12), 71 % des fonctions (5/7), aucune branche**. Le chiffre est flatteur : la base de code est minuscule et les tests ne vérifient que l'instanciation des classes, aucun comportement. Non couverts : `getOlympics()` et son `catchError`.

Anomalie détectée : en cas d'erreur HTTP, `catchError((error, caught) => caught)` se ré-abonne au même flux : la requête est relancée en boucle et la page ne s'affiche jamais. Ce cas doit être couvert par un test et corrigé (par exemple renvoyer une liste vide et afficher un message).

Workshop Organizer : 3 classes, 2 tests exécutés, tous réussis

Classe de test	Ce qui est vérifié	Ce qui n'est pas vérifié
`NotionServiceTest`	`createNotion` : création d'une notion, nom restitué (dépôt simulé par Mockito)	`getAll()`
`WorkshopServiceTest`	`createWorkshop` : création d'un atelier avec 2 notions, identifiant renvoyé	`getById()` (dont le cas « inexistant »), `getAll()`
`JavaBasicAppApplicationTests`	Annotée `@SpringBootTest` mais sans méthode : 0 test exécuté	Chargement du contexte Spring

Couverture mesurée avec JaCoCo (absent du projet, ajouté temporairement pour l'analyse, code généré exclu) : **43 % des lignes (22/51), 37 % des instructions, 0 % des branches (0/2)**. Détail : services 44 % (8/18 lignes), contrôleurs 0 %, configuration `WebMvcConfig` 0 %, entités 100 % (accesseurs appelés indirectement par ModelMapper).

Objectifs

- Tester le **comportement** et non la seule instanciation : rendu des données, cas vides, cas d'erreur.
- Couvrir toutes les méthodes publiques des services Java, y compris la branche « non trouvé » de `getById()`.
- Tester les contrôleurs isolément (`@WebMvcTest` + MockMvc, service simulé) : codes HTTP et JSON produit.
- Tests à ajouter en priorité : `OlympicService.getOlympics()` (succès et erreur, avec `HttpTestingController`), `HomeComponent` (affichage de N pays et de leurs participations), `WorkshopService.getById()` et `getAll()`, `NotionService.getAll()`, `NotionController`,

`WorkshopController`.

2.2 Outils et technologies

Application	Outil	Version	Rôle
Olympic	Jasmine (`jasmine-core`)	4.2.0	Framework de test : suites, assertions, espions
Olympic	Karma	6.4.4	Lanceur des tests dans un vrai navigateur
Olympic	karma-chrome-launcher / Chrome Headless	3.1	Navigateur sans interface ; options `--no-sandbox` pour les conteneurs CI (`karma.conf.js`)
Olympic	Angular TestBed (`@angular/core`)	20.3.7	Configuration des composants et services testés
Olympic	karma-junit-reporter	2.0.1	Rapport JUnit XML exploité par GitHub Actions
Olympic	karma-coverage (Istanbul)	2.2.1	Couverture de code (HTML et résumé texte)
Olympic	Node.js / npm	22	Exécution ; en CI `setup-node` avec cache npm
Workshop	JUnit Jupiter (via `spring-boot-starter-test`)	5.10.2	Framework de test
Workshop	Mockito	5.7.0	Simulation des dépôts (`@Mock`, `@InjectMocks`, `@Spy`)
Workshop	Spring Test (`SpringExtension`)	6.1 (Boot 3.2.4)	Intégration JUnit / Spring
Workshop	Gradle (wrapper)	8.7	Tâche `test` sur JUnit Platform, rapports XML et HTML ; en CI `setup-gradle` avec cache
Workshop	Java (Temurin)	21	Exécution
Workshop	JaCoCo — recommandé	0.8.x	Couverture et seuil bloquant (`jacocoTestCoverageVerification`)

Justification : ce sont les outils par défaut des écosystèmes Angular et Spring Boot, déjà intégrés aux projets, sans coût de migration. Karma est déprécié par l'équipe Angular : une migration vers **Jest** ou **Vitest** (exécution sans navigateur, plus rapide) est à planifier, sans urgence.

2.3 Critères de réussite

Critère	Seuil	Justification
Taux de réussite	100 % des tests réussis	Un test en échec bloque le job `test`, donc la construction et la publication de l'image (déjà en place : `build` dépend de `test`).
Couverture lignes / instructions	≥ 80 % par application	Standard courant (Google qualifie 75 % de « louable » et 90 % d'« exemplaire ») ; atteignable rapidement vu la taille du code.
Couverture des branches	≥ 70 %	Les branches portent les cas d'erreur (atelier inexistant, échec HTTP), les plus exposés aux régressions.
Logique métier (services Java, service Angular)	≥ 90 % des lignes	Code à plus forte valeur et le plus simple à tester isolément.
Nouveau code d'une PR	≥ 80 % ; aucun test supprimé sans justification	Empêche la dégradation progressive de la couverture.
Durée	< 2 min par application en CI	Retour rapide aux développeurs.
Stabilité	0 test instable toléré	Un test instable est corrigé ou mis en quarantaine sous 48 h, sinon on cesse de faire confiance aux résultats.

Exclus de la mesure : code généré (`src/main/java-generated`), classe de démarrage

`JavaBasicAppApplication`, types TypeScript sans logique, fichiers d'environnement.

Écart actuel : front 75 %, back 43 %, tous deux sous le seuil. Le seuil est d'abord appliqué en avertissement avec un palier à 70 %, puis rendu bloquant à 80 % une fois les tests listés en 2.1 ajoutés.

Mise en œuvre : `coverageReporter.check.global` de karma-coverage (`statements: 80`, `branches: 70`) côté Angular ; plugin `jacoco` et tâche `jacocoTestCoverageVerification` rattachée à `check` côté Gradle.

2.4 Commandes d'exécution et rapports

Les deux applications (script du dépôt, utilisé par la CI)

```
./run-tests.sh                          # les deux applications
./run-tests.sh olympic-games-starter    # front seul
./run-tests.sh workshop-organizer       # back seul
```

Le script détecte le type de projet (`package.json`, ou `build.gradle` + `gradlew`), lance les tests et copie les rapports JUnit XML dans `test-results/<application>/` à la racine du dépôt. Code retour 0 si tout réussit, 1 sinon.

Olympic Participation Tracker

```
cd olympic-games-starter
npm ci
npm test                                  # = ng test --watch false (ChromeHeadless)
npx ng test --watch=false --code-coverage  # avec couverture
```

Workshop Organizer

```
cd workshop-organizer
./gradlew clean test                      # Windows : gradlew.bat clean test
```

Aucune base de données n'est nécessaire : les tests unitaires simulent les dépôts JPA.

Localisation des rapports

Rapport	Emplacement	Format
Front : résultats	`olympic-games-starter/reports/*.xml`	JUnit XML
Front : couverture	`olympic-games-starter/coverage/olympic-games-starter/index.html`	HTML Istanbul + résumé console
Back : résultats	`workshop-organizer/build/test-results/test/TEST-*.xml`	JUnit XML
Back : rapport lisible	`workshop-organizer/build/reports/tests/test/index.html`	HTML Gradle
Back : couverture (après ajout de JaCoCo)	`workshop-organizer/build/reports/jacoco/test/html/index.html`	HTML + XML
Consolidé	`test-results/<application>/*.xml` (copie par `run-tests.sh`)	JUnit XML
GitHub Actions	Onglet Checks : « Rapport de tests - <application> » (annotations sur les tests en échec, résumé détaillé) ; artefact `test-results-<application>` conservé 14 jours	Publié par `mikepenz/action-junit-report`

2.5 Fréquence d'exécution

- **En local** : avant chaque commit ou push (`./run-tests.sh <application>`), et en continu pendant le

développement (`ng test` en mode watch).

- **En CI, à chaque push sur toutes les branches et à chaque pull request vers** `main` : job `test` de `ci.yml`, pour l'application modifiée uniquement (filtre par chemins), pour les deux si le script de test ou le workflow change. Lancement manuel possible (`workflow_dispatch`).
- **Bloquant** : un échec empêche la construction et la publication de l'image Docker. Une règle de protection de `main` (« Require status checks to pass ») est à activer pour bloquer aussi la fusion.

Responsabilités : l'auteur du commit (Dev) corrige immédiatement tout test en échec ; le relecteur de la PR vérifie que le nouveau code est accompagné de tests ; le responsable du pipeline (Ops) surveille la durée et la stabilité des suites.

3. Tests d'intégration

Les tests d'intégration vérifient que les composants fonctionnent ensemble avec leurs vraies dépendances : contrôleur, service, JPA et PostgreSQL pour l'API ; routeur, resolver, service HTTP et composants pour le front ; chaque image Docker dans son serveur d'exécution. Ils ne sont pas encore implémentés, à l'exception du test de fumée des images dans le pipeline.

3.1 Périmètre et objectifs

Workshop Organizer (priorité haute : persistance et contrat d'API)

- Chaîne complète HTTP → contrôleur → service → dépôt JPA → PostgreSQL réel.
- Conformité des réponses au contrat `openapi.yaml` (codes HTTP, schémas JSON).
- Relation n-n atelier / notions (table `workshop_model_notions`, cascade MERGE).
- Validation des entrées (`@NotNull` générés) et format des erreurs (schéma `Error`).
- Compatibilité du schéma Hibernate (`ddl-auto=update`) avec le script `db/00001_0.0.0_init_create.sql`.
- Configuration par variables d'environnement (URL, utilisateur, mot de passe lu par `entrypoint.sh`).

Olympic Participation Tracker

- Navigation réelle : route `''` → resolver → `OlympicService` → `HomeComponent` alimenté par `withComponentInputBinding`.
- Route inconnue → `NotFoundComponent`.
- Comportement en cas d'échec du chargement du JSON (boucle de relance signalée en 2.1).
- Image Nginx : service de l'application, repli SPA (`try_files … /index.html`), types MIME et en-têtes de cache.

Objectif : détecter avant la construction de l'image les défauts invisibles en isolation (mappage ModelMapper, SQL généré, configuration Spring, câblage du routeur).

3.2 Outils et technologies recommandés

Outil	Application	Usage	Justification
Spring Boot Test (`@SpringBootTest`, `@WebMvcTest`), MockMvc, `TestRestTemplate`	Workshop	Démarrer le contexte Spring et appeler les endpoints	Déjà fourni par `spring-boot-starter-test` : aucune dépendance supplémentaire.
Testcontainers (module PostgreSQL) + `@ServiceConnection`	Workshop	Base PostgreSQL 13 jetable pour chaque suite	Même moteur et même version qu'en production, contrairement à H2 ; fonctionne sur les runners GitHub (Docker disponible) ; `@ServiceConnection` (Spring Boot ≥ 3.1) configure la datasource.
swagger-request-validator (module MockMvc)	Workshop	Valider chaque requête et réponse contre `openapi.yaml`	Le contrat existe déjà : il devient l'oracle des tests.
TestBed + `provideRouter(routes)` + `provideHttpClientTesting` / `HttpTestingController`	Olympic	Intégration routeur / resolver / service / composant sans serveur	API officielle Angular ; s'exécute dans la suite Karma existante.
Docker + curl (job `build` de `ci.yml`), éventuellement Hurl	Les deux	Tests de l'image construite	Prolonge le test de fumée déjà en place, sans nouvel outil.

Environnement de test : runner `ubuntu-latest` avec Docker ; conteneur `postgres:13` lancé par Testcontainers (identique en local) ; profil Spring `test` (`spring.jpa.show-sql=false`, schéma initialisé par le script SQL du dépôt) ; aucune donnée ni secret de production, le mot de passe étant généré par Testcontainers.

Intégration au pipeline : nouveau job `integration` dans `ci.yml`, exécuté après `test` et requis par `build`. Tests séparés par la convention `*IT.java` et une tâche Gradle `integrationTest` (ou un tag JUnit `@Tag("integration")`).

3.3 Scénarios de test prioritaires

Workshop Organizer

ID	Scénario	Résultat attendu
TI-B01	`POST /api/notions` `{"name":"Docker"}` puis `GET /api/notions`	201 Created (contrat) ; notion présente dans la liste et en base
TI-B02	`POST /api/workshops` avec 2 notions existantes, puis `GET /api/workshops/{id}`	201, identifiant généré, les 2 notions renvoyées ; 2 lignes dans `workshop_model_notions`
TI-B03	`POST /api/workshops` avec une notion inexistante	Comportement défini et documenté (création par cascade ou 400), jamais de 500
TI-B04	`GET /api/workshops/9999` (inexistant)	404 au format `Error`. **Échoue aujourd'hui** : l'API renvoie 200 avec un corps vide
TI-B05	`POST` sans `name`, JSON mal formé, `name` de plus de 255 caractères	400 au format `Error`, rien n'est enregistré (pas de 500 sur la colonne `varchar(255)`)
TI-B06	`POST /api/notions` avec un nom déjà existant	409 Conflict. **Aujourd'hui** `save()` écrase silencieusement (le nom est la clé primaire)
TI-B07	`GET /api/workshops?limit=1` puis `limit=101`	Au plus 1 élément ; 400 au-delà de 100 (contrat). **Paramètre ignoré aujourd'hui**
TI-B08	Démarrage sur une base initialisée par `db/00001_0.0.0_init_create.sql`	L'application démarre ; `GET /api/workshops` renvoie « Workshop 1 » avec la notion « Docker »
TI-B09	Base indisponible au démarrage	Échec rapide et explicite ; avec Actuator, `/actuator/health` renvoie DOWN

Olympic Participation Tracker

ID	Scénario	Résultat attendu
TI-F01	Navigation vers `/`, réponse simulée de 3 pays	Une seule requête vers `assets/mock/olympic.json` ; « 3 data loaded », 3 pays, une ligne par participation (ville, année, athlètes, médailles)
TI-F02	Réponse HTTP 404 ou 500 sur le JSON	Aucune nouvelle requête après l'erreur (`HttpTestingController.verify()`), message affiché. **Échoue aujourd'hui** (relance en boucle)
TI-F03	Réponse `[]`	« 0 data loaded », aucun pays, aucune erreur
TI-F04	Navigation vers `/inconnu`	`NotFoundComponent` affiché
TI-F05	Image Nginx : `GET /`, `GET /une/route/profonde`, `GET /assets/mock/olympic.json`	200 + `<app-root>` ; 200 + `index.html` (repli SPA) ; 200 + `Content-Type: application/json`

Données de test : jeu minimal versionné, celui du script SQL (`Docker`, `Workshop 1`), complété par des constructeurs de données dans les tests ; pour le front, une fixture JSON de 3 pays dont un sans

participation. Chaque test part d'une base propre (transaction annulée ou `TRUNCATE`) pour ne pas dépendre de l'ordre d'exécution.

Les scénarios marqués « échoue aujourd'hui » documentent des écarts avec le contrat ; une fois les corrections faites, ils servent de tests de non-régression.

3.4 Critères de réussite

Critère	Seuil	Justification
Scénarios P1 (TI-B01 à TI-B05, TI-F01, TI-F02)	100 % réussis	Ils portent la valeur métier ; bloquant pour le job `build`.
Couverture du contrat	Chacune des 5 opérations d'`openapi.yaml` : ≥ 1 cas nominal + ≥ 1 cas d'erreur	Garantit que le contrat publié est réellement respecté.
Conformité au contrat	0 écart signalé par le validateur OpenAPI	Les clients de l'API s'appuient sur le contrat.
Erreurs serveur	0 réponse 5xx sur une entrée invalide	Une entrée invalide est une erreur du client (4xx).
Couverture unitaire + intégration (back)	≥ 85 % des lignes, contrôleurs inclus	Les contrôleurs ne se testent utilement qu'en intégration.
Durée	< 5 min par application	Conteneur PostgreSQL partagé par toute la suite.
Reproductibilité	0 test instable ; aucun prérequis local hors Docker	Même résultat en local et en CI.

3.5 Fréquence d'exécution recommandée

- **À chaque pull request et à chaque push sur** `main` : job `integration`, bloquant, pour l'application modifiée.
- **Branches de développement** : à chaque push tant que la durée reste sous 5 min, sinon à la demande (`workflow_dispatch`).
- **En local** : avant d'ouvrir une PR qui touche l'API, le modèle JPA ou le routeur (`./gradlew integrationTest`).
- **Avant chaque release** : suite complète sur les deux applications.

Responsables : les Dev écrivent et maintiennent les tests avec chaque fonctionnalité ; la QA définit et relit les scénarios (cas limites, données) ; l'Ops maintient l'environnement (Docker, Testcontainers, durée du job). Un échec bloque la fusion et l'auteur de la PR le corrige.

4. Tests de performance

Les tests de performance vérifient les temps de réponse, la capacité et la stabilité sous charge. Ils ne sont pas implémentés ; la stratégie ci-dessous est à appliquer sur un environnement de pré-production identique à la production (mêmes images GHCR, mêmes ressources).

4.1 Périmètre et objectifs

Workshop Organizer (cible principale)

- `GET /api/workshops` : endpoint le plus exposé. La relation n-n est chargée en `EAGER` et la liste n'est pas paginée (`limit` ignoré) : risque de requêtes N+1 et de temps de réponse proportionnel au volume.
- `GET /api/workshops/{id}` et `GET /api/notions` : lectures simples.
- `POST /api/workshops` et `POST /api/notions` : écritures, table de jointure, contention éventuelle.
- Ressources : CPU et mémoire de la JVM, pool de connexions Hikari (10 par défaut), temps des requêtes PostgreSQL.

Olympic Participation Tracker

- Page d'accueil `/` : chargement initial (bundle JavaScript + JSON) et Core Web Vitals.
- Serveur Nginx : débit et latence des fichiers statiques sous forte concurrence.
- Taille des bundles (budgets Angular).

Objectifs : établir une référence, vérifier les seuils de 4.3 à chaque version, connaître le point de rupture et détecter les fuites mémoire.

4.2 Outils et technologies recommandés

Outil	Application	Usage	Justification
k6 (Grafana)	Workshop + Nginx	Scénarios de charge écrits en JavaScript ; seuils déclaratifs (`thresholds`) qui font échouer l'exécution	Scripts versionnés avec le code ; léger (binaire ou image `grafana/k6`) ; s'exécute dans GitHub Actions ; plus simple à maintenir que JMeter (XML, interface graphique).
Lighthouse CI (`@lhci/cli`)	Olympic	Core Web Vitals, score de performance, assertions bloquantes	Référence de Google pour les métriques web, intégrable en CI.
Budgets Angular (`angular.json`)	Olympic	Taille du bundle initial à chaque build	Déjà configurés : avertissement à 500 ko, erreur à 1 Mo.
Spring Boot Actuator + Micrometer, Prometheus / Grafana	Workshop	Métriques JVM, HTTP et pool de connexions pendant les tirs	Relie la latence observée à sa cause (GC, pool saturé, SQL).
`pg_stat_statements` (PostgreSQL)	Workshop	Requêtes SQL les plus coûteuses	Met en évidence le N+1 de la relation EAGER.

Configuration recommandée : pré-production (namespace Kubernetes dédié ou `docker compose`) déployée avec l'image du commit testé et des ressources identiques à la production ; base pré-remplie (200 notions, 1 000 ateliers de 5 notions en moyenne) ; générateur de charge sur une machine distincte de l'application ; minute de chauffe de la JVM exclue des mesures ; résultats (JSON k6, rapports Lighthouse) archivés comme artefacts du workflow.

4.3 Métriques et seuils recommandés

Métrique	Périmètre	Seuil	Justification
Latence p95 des lectures	`GET` API	< 300 ms	Lectures simples sur une petite base ; marge sous la seconde, où l'utilisateur perçoit une attente.
Latence p99 des lectures	`GET` API	< 800 ms	Borne les cas extrêmes (GC, cache froid).
Latence p95 des écritures	`POST` API	< 500 ms	Transaction et table de jointure.
Taux d'erreur	API, charge nominale	< 1 % ; 0 réponse 5xx	Des erreurs sous charge révèlent une saturation (pool, mémoire).
Débit soutenu	API	≥ 50 requêtes/s à 20 utilisateurs virtuels	Usage attendu d'un outil interne, avec une marge ×5.
CPU / mémoire	Conteneur API	CPU < 70 % ; tas JVM < 80 % après GC	Marge pour les pics et le ramasse-miettes.
Dérive en endurance	API	p95 et mémoire stables à ± 10 % sur 2 h	Détecte les fuites et les dégradations progressives.
Latence des fichiers statiques	Nginx	p95 < 100 ms à 200 utilisateurs virtuels	Contenu statique compressé : doit rester quasi instantané.
Score Lighthouse performance	Front	≥ 90 (bureau), ≥ 80 (mobile)	Seuil « bon » de Lighthouse.
LCP / CLS / TBT	Front	LCP < 2,5 s ; CLS < 0,1 ; TBT < 200 ms	Seuils « bons » des Core Web Vitals (web.dev).
Bundle initial	Front	< 500 ko (avertissement) ; < 1 Mo (bloquant)	Budgets déjà présents dans `angular.json`.

Ces seuils sont des valeurs de départ. Ils seront ajustés après une première campagne de référence sur l'environnement cible ; ensuite, toute dégradation de plus de 20 % par rapport à la référence est traitée comme un échec.

4.4 Scénarios de charge

Scénario	Paramètres	Objectif
TP-01 Fumée de performance	2 utilisateurs virtuels (VU), 1 min, tous les endpoints	Valider les scripts, détecter une régression grossière ; exécutable en CI
TP-02 Charge nominale	20 VU, 10 min ; 80 % de lectures (`GET /workshops`, `/workshops/{id}`, `/notions`), 20 % d'écritures (`POST`) ; pause de 1 à 3 s entre deux requêtes	Vérifier les seuils de 4.3 en usage normal
TP-03 Montée en charge (stress)	Paliers 10 → 50 → 100 → 200 VU, 3 min par palier	Trouver le point de rupture (p95 > 1 s ou erreurs > 5 %) et la ressource limitante
TP-04 Pic soudain (spike)	10 → 150 VU en 10 s, maintien 1 min, retour à 10	Absence d'erreurs en cascade ; retour à la normale en moins d'1 min
TP-05 Endurance (soak)	20 VU pendant 2 h	Fuites mémoire, épuisement du pool de connexions, dérive de latence
TP-06 Volume	`GET /api/workshops` avec 10 000 ateliers en base	Mesurer l'effet de l'absence de pagination et du chargement EAGER
TP-07 Front	Lighthouse CI (3 passages, médiane) sur `/` ; k6 à 200 VU sur Nginx (`/`, bundle JS, JSON)	Core Web Vitals et tenue du serveur statique

Usage de référence : un outil interne utilisé par quelques dizaines de personnes en même temps (20 VU en nominal), dimensionné avec une marge de ×5 à ×10 pour les pics.

4.5 Fréquence d'exécution recommandée

- **TP-01** : à chaque fusion sur `main` (1 min) ; non bloquant au début, bloquant une fois la référence stable.
- **Budgets Angular** : à chaque build (déjà actif).
- **TP-02 et TP-07** : chaque semaine (déclenchement `schedule` du workflow) sur la pré-production, et avant chaque release.
- **TP-03 à TP-06** : avant une release majeure, après un changement d'infrastructure (ressources Kubernetes, version de PostgreSQL ou de la JVM) ou du modèle de données ; au moins une fois par trimestre.

Responsables : l'Ops (ou une QA performance) prépare l'environnement, lance les campagnes et compare à la référence ; l'analyse se fait avec le Dev référent de l'application. Le tech lead décide des actions correctives (optimisation de requête, pagination, dimensionnement) ou d'un report de mise en production, avec le PO si l'impact est fonctionnel.

5. Tests de sécurité

Les tests de sécurité recherchent les vulnérabilités du code, des dépendances, des images et des applications en fonctionnement. Seul le scan de secrets (Gitleaks) est implémenté ; le reste est recommandé avant une mise en production.

5.1 Périmètre et objectifs

- **Secrets** : aucun identifiant dans le dépôt, l'historique git ou les logs du pipeline. Déjà contrôlé par Gitleaks ; le mot de passe de la base vient du secret GitHub `CI_DB_PASSWORD` en CI et d'un secret Docker en local.
- **Code** : analyse statique Java et TypeScript (injections, mauvaises pratiques).
- **Dépendances** : bibliothèques npm et Gradle. Spring Boot 3.2.4 n'est plus maintenu en open source : montée de version à prévoir.
- **Images Docker et manifestes** : images de base `nginx:1.27-alpine` et `eclipse-temurin:21-jre` ; conteneurs exécutés en root (pas de `USER` dans les Dockerfile) ; manifestes `k8s/`.
- **API en fonctionnement** : validation des entrées, codes d'erreur, absence de fuite d'information, consommation de ressources.
- **Front en fonctionnement** : en-têtes de sécurité HTTP servis par Nginx, absents aujourd'hui (CSP, `X-Content-Type-Options`, `X-Frame-Options`, HSTS derrière TLS) ; version du serveur exposée.

Point d'attention : l'API n'a **aucune authentification**, n'importe qui peut créer des notions et des ateliers. Ce n'est pas testable en l'état (section 7), mais c'est le premier risque à traiter avant une exposition publique.

5.2 Outils et technologies recommandés

Outil	Type	Intégration	Justification
Gitleaks 8.24 — **implémenté**	Secrets	Job `secrets-scan` de `ci.yml` : historique complet, valeurs masquées (`--redact`), bloquant	Rapide, sans compte externe, couvre tout l'historique git.
CodeQL (GitHub code scanning)	Analyse statique (SAST) Java + TypeScript	`github/codeql-action` sur les PR, les push sur `main` et chaque semaine ; résultats SARIF dans l'onglet Security	Natif GitHub, gratuit pour un dépôt public, suit les flux de données (injections).
Dependabot, `npm audit`, OWASP Dependency-Check (plugin Gradle)	Dépendances (SCA)	Alertes et PR de mise à jour automatiques ; contrôle bloquant sur les PR qui modifient `package-lock.json` ou `build.gradle`	Couvre les deux écosystèmes avec une base de CVE à jour.
Trivy	Images, Dockerfile, manifestes Kubernetes	Dans le job `build`, entre le build local et le push GHCR : bloque la publication	Un seul outil pour l'image et l'infrastructure ; s'insère dans le pipeline Docker existant.
Hadolint	Dockerfile	Lint sur les PR qui modifient un Dockerfile	Bonnes pratiques : utilisateur non root, versions figées.
OWASP ZAP (baseline + API scan)	Analyse dynamique (DAST)	Scan passif du front ; scan actif de l'API piloté par `openapi.yaml`, sur la pré-production	Teste l'application réelle ; l'import du contrat couvre tous les endpoints.

Principe : les contrôles rapides (secrets, SAST, dépendances, image) s'exécutent dans le pipeline à chaque modification et bloquent la publication ; le DAST, plus long et qui exige une application déployée, tourne sur la pré-production. Tous les résultats sont centralisés dans l'onglet Security de GitHub (format SARIF).

5.3 Vulnérabilités à vérifier en priorité

Catégorie (OWASP)	Risque dans ces applications	Vérification
API2 / A07 : authentification absente	Tous les endpoints d'écriture sont publics	Revue d'architecture ; tests à écrire dès l'ajout d'une authentification
API4 : consommation de ressources non bornée	Liste des ateliers non paginée (`limit` ignoré), pas de limite de taille de requête ni de débit	TI-B07, TP-06, ZAP ; limitation de débit à prévoir (ingress ou passerelle)
A03 : injection (SQL / JPQL, XSS)	Noms de notions et d'ateliers stockés puis renvoyés	Requêtes JPA paramétrées à confirmer par CodeQL ; charges d'injection via ZAP ; échappement Angular par défaut (pas d'`innerHTML`)
API3 : affectation de masse	Un client envoie un `id` ou des champs imprévus ; ModelMapper copie les propriétés par nom	Test d'intégration : un `id` fourni dans un `POST` est ignoré
A04 : validation des entrées	Seul `name` est `@NotNull` ; aucune longueur maximale alors que la colonne est en `varchar(255)`	TI-B05 ; ajout de `maxLength` dans le contrat
A05 : mauvaise configuration	Swagger UI public, `spring.jpa.show-sql=true` (requêtes dans les logs), en-têtes HTTP absents, conteneurs en root	ZAP baseline, Trivy config, Hadolint ; revue du profil Spring de production
A06 : composants vulnérables	Spring Boot 3.2.4 hors maintenance, RxJS 7.5, dépendances transitives	Dependabot, Dependency-Check, `npm audit`, Trivy
A02 / A08 : secrets et intégrité	Mot de passe de la base, `k8s/secret.yaml` versionné, actions GitHub référencées par tag	Gitleaks (actif) ; secrets via GitHub Secrets ou un coffre externe ; actions épinglées par SHA
A09 : journalisation	Logs SQL bavards ; aucune trace des créations	Revue des logs produits pendant le DAST

Références : OWASP Top 10 2021 (A01 à A10) et OWASP API Security Top 10 2023 (API1 à API10).

5.4 Critères de réussite recommandés

Contrôle	Seuil de réussite (bloquant)
Secrets (Gitleaks)	0 secret détecté. **Implémenté et bloquant.**
Analyse statique (CodeQL)	0 alerte de sévérité élevée ou critique non traitée
Dépendances et images (Dependabot, Dependency-Check, Trivy)	0 vulnérabilité CRITICAL ; 0 HIGH disposant d'un correctif ; les MEDIUM sont suivies dans un ticket
Analyse dynamique (ZAP)	0 alerte High ; alertes Medium corrigées ou acceptées par écrit
Configuration	Conteneurs non root, en-têtes de sécurité présents, Swagger UI et logs SQL désactivés en production

Délais de correction selon la criticité (score CVSS) :

Criticité	Délai de correction	Exemple
Critique (CVSS ≥ 9) ou secret exposé	24 h ; un secret exposé est révoqué et remplacé immédiatement (< 1 h), puis purgé de l'historique	Mot de passe commité, exécution de code à distance dans une dépendance
Élevée (7 à 8,9)	7 jours	Injection exploitable, image de base vulnérable
Moyenne (4 à 6,9)	30 jours	En-tête de sécurité manquant
Faible (< 4)	Prochaine release (90 jours maximum)	Version du serveur exposée

Toute exception (risque accepté) est justifiée, validée par le responsable sécurité et réexaminée à chaque release.

5.5 Fréquence d'exécution recommandée

- **À chaque push et PR** : scan de secrets (actif), analyse statique CodeQL.
- **À chaque PR qui modifie les dépendances, et chaque jour** (Dependabot) : analyse des dépendances.
- **À chaque build d'image** : Trivy avant le push sur GHCR.
- **Chaque semaine et avant chaque release** : ZAP sur la pré-production ; CodeQL planifié pour bénéficier des nouvelles règles.
- **Avant la première mise en production, puis chaque année** : test d'intrusion manuel.

Responsables : le DevSecOps maintient les outils, trie les alertes et suit les délais ; les Dev corrigent le code et les dépendances ; l'Ops renouvelle les secrets et met à jour les images de base ; le tech lead arbitre les risques acceptés.

6. Fonctionnalités à tester

Fonctionnalités retenues, classées par priorité selon le risque (impact pour l'utilisateur × probabilité de régression). **P1** : bloquant pour une release ; **P2** : important ; **P3** : secondaire.

ID	Application	Fonctionnalité	Raison / risque	Tests	Priorité
F-01	Olympic	Chargement des données (resolver + `OlympicService.getOlympics()`)	Point d'entrée unique des données ; boucle de relance en cas d'erreur	Unitaire ; intégration TI-F01, TI-F02	P1
F-02	Olympic	Affichage : pays, participations (ville, année, athlètes, médailles), compteur « N data loaded »	Valeur métier de l'application	Unitaire ; intégration TI-F01, TI-F03	P1
F-03	Olympic	Gestion de l'échec de chargement (JSON absent ou invalide)	Aujourd'hui : page bloquée et requêtes en boucle	Unitaire ; intégration TI-F02	P1
F-04	Olympic	Routage et page 404 (route `**`, repli `try_files` de Nginx)	Une URL directe ne doit pas renvoyer d'erreur serveur	Intégration TI-F04, TI-F05	P2
F-05	Olympic	Service de l'application par l'image Nginx	Condition de toute mise en production	Smoke test CI (implémenté) ; TP-07 ; en-têtes de sécurité	P1
F-06	Workshop	Création d'une notion (`POST /api/notions`)	Écriture ; le nom est la clé primaire (risque d'écrasement)	Unitaire (existant) ; TI-B01, TI-B06	P1
F-07	Workshop	Liste des notions (`GET /api/notions`)	Lecture simple	Unitaire ; intégration ; smoke test CI (implémenté)	P2
F-08	Workshop	Création d'un atelier avec ses notions (`POST /api/workshops`)	Relation n-n, cascade, mappage ModelMapper	Unitaire (existant) ; TI-B02, TI-B03	P1
F-09	Workshop	Consultation d'un atelier (`GET /api/workshops/{id}`)	Cas « inexistant » non conforme (200 vide au lieu de 404)	Unitaire ; TI-B04	P1
F-10	Workshop	Liste des ateliers et paramètre `limit`	Liste non paginée : risque de performance	TI-B07 ; TP-02, TP-06	P2
F-11	Workshop	Validation des entrées et format des erreurs	Possibles 500 sur entrée invalide ; injection	TI-B05 ; ZAP	P1
F-12	Workshop	Persistance PostgreSQL et schéma (script SQL + `ddl-auto=update`)	Dérive de schéma entre environnements	TI-B08	P1
F-13	Workshop	Configuration par variables d'environnement et secret (mot de passe lu par `entrypoint.sh`)	Différences local / CI / Kubernetes ; fuite de secret	Smoke test CI (implémenté) ; TI-B09 ; Gitleaks	P1
F-14	Les deux	Construction et publication des images (tags branche-sha)	Une image défectueuse ne doit jamais atteindre GHCR	Smoke test avant push (implémenté) ; Trivy	P1
F-15	Olympic	Page 404 : message et lien « Go back »	Lien inactif : `RouterLink` non importé (composant non standalone déclaré dans aucun module)	Unitaire (lien vers `/`)	P3

7. Fonctionnalités à ne pas tester

Élément exclu	Justification
Code généré par OpenAPI Generator (`src/main/java-generated` : interfaces `NotionsApi`, `WorkshopsApi`, DTO)	Régénéré à chaque build depuis `openapi.yaml` et éprouvé par l'outil ; exclu de la couverture. Son comportement est vérifié indirectement par les tests d'intégration et de contrat.
Accesseurs des entités JPA, types TypeScript `Olympic` et `Participation`	Aucune logique ; déjà exercés indirectement par les autres tests.
Fonctionnement interne des frameworks et bibliothèques (Angular, Spring Data JPA, Hibernate, ModelMapper, Nginx, PostgreSQL)	Testés par leurs mainteneurs ; seule notre utilisation (configuration, mappage, requêtes) est testée.
Fichiers Swagger UI (`src/main/resources/static`)	Composant tiers statique ; la décision de sécurité est de le désactiver en production plutôt que de le tester.
Authentification, autorisation, sessions, rôles	Non implémentés : rien à tester. Le risque est signalé en 5.1 ; les tests seront écrits avec la fonctionnalité.
Intégration front Olympic / API Workshop, tests de bout en bout croisés	Les deux applications sont indépendantes (le front lit un JSON local) ; à revoir si le front consomme un jour une API.
Publication Maven vers GitLab (bloc `publishing` de `build.gradle`) et `npm pack`	Non utilisés : la livraison passe par les images Docker publiées sur GHCR.
Compatibilité multi-navigateurs, responsive, accessibilité exhaustive	Interface de prototype sans maquette ; tests limités à Chrome Headless. Le score d'accessibilité Lighthouse est relevé à titre indicatif ; une campagne Playwright multi-navigateurs sera ajoutée quand l'interface sera stabilisée.
Fichiers d'environnement Angular (`environment*.ts`)	Constantes sans logique.
Déploiement Kubernetes (manifestes `k8s/`)	Hors périmètre de cette étape : seule l'analyse de configuration par Trivy est prévue (5.2) ; le déploiement sera validé par des tests de fumée post-déploiement dans une étape ultérieure.

8. Matrice récapitulative des tests

Synthèse de la stratégie : une ligne par type de test, en distinguant les contrôles **implémentés** dans le pipeline `ci.yml` de ceux **recommandés** pour une mise en production.

Type de test	Outil(s)	Fréquence	Responsable (Dev / QA / Ops / DevSecOps)	Critères de réussite	Statut (Implémenté / Recommandé)
Unitaires : front	Jasmine 4.2, Karma 6.4 (Chrome Headless), karma-junit-reporter, karma-coverage	Chaque push et PR (job `test`) ; en local avant commit	Dev	100 % réussis ; couverture ≥ 80 % lignes, ≥ 70 % branches	Implémenté (seuil de couverture : recommandé)
Unitaires : back	JUnit 5.10, Mockito 5.7, Gradle 8.7 ; JaCoCo	Chaque push et PR (job `test`) ; en local avant commit	Dev	100 % réussis ; couverture ≥ 80 % lignes (services ≥ 90 %), ≥ 70 % branches	Implémenté (JaCoCo : recommandé)
Smoke test des images Docker	`docker run` + curl, PostgreSQL 13 éphémère (job `build`)	Chaque build, avant le push GHCR	Ops	Front : HTTP 200 + `<app-root>` en ≤ 30 s ; back : `/api/notions` 200 en ≤ 90 s ; sinon pas de publication	Implémenté
Scan des secrets	Gitleaks 8.24 (historique complet)	Chaque push et PR	DevSecOps	0 secret détecté (bloquant)	Implémenté
Intégration : back	Spring Boot Test, MockMvc, Testcontainers PostgreSQL 13, swagger-request-validator	Chaque PR et push sur `main` ; avant release	Dev (scénarios : QA)	100 % des scénarios P1 ; 5 opérations OpenAPI couvertes ; 0 écart au contrat ; < 5 min	Recommandé
Intégration : front	TestBed + Router + HttpTestingController ; tests du conteneur Nginx	Chaque PR et push sur `main`	Dev	TI-F01 à TI-F05 réussis ; 1 seule requête par chargement	Recommandé
Bout en bout (E2E)	Playwright	Chaque fusion sur `main` (parcours critiques) ; suite complète avant release	QA	100 % des parcours critiques réussis	Recommandé
Performance : API	k6, Actuator / Prometheus	TP-01 à chaque fusion sur `main` ; TP-02 chaque semaine ; TP-03 à TP-06 avant release majeure	Ops (analyse : Dev)	p95 GET < 300 ms, POST < 500 ms ; erreurs < 1 % ; 0 réponse 5xx	Recommandé
Performance : front	Lighthouse CI, budgets Angular	Budgets à chaque build ; Lighthouse chaque semaine et avant release	Dev / QA	Score ≥ 90 ; LCP < 2,5 s ; bundle initial < 1 Mo	Budgets : implémentés ; Lighthouse : recommandé
Sécurité : analyse statique	CodeQL	Chaque PR, push sur `main`, chaque semaine	DevSecOps	0 alerte élevée ou critique	Recommandé
Sécurité : dépendances	Dependabot, `npm audit`, OWASP Dependency-Check	Chaque PR sur les dépendances ; chaque jour	DevSecOps	0 CRITICAL ; 0 HIGH corrigeable	Recommandé

Type de test	Outil(s)	Fréquence	Responsable (Dev / QA / Ops / DevSecOps)	Critères de réussite	Statut (Implémenté / Recommandé)
Sécurité : images et IaC	Trivy, Hadolint	Chaque build d'image	DevSecOps / Ops	0 CRITICAL ; 0 HIGH corrigeable ; conteneurs non root	Recommandé
Sécurité : analyse dynamique	OWASP ZAP (baseline + API scan)	Chaque semaine et avant release	DevSecOps	0 alerte High	Recommandé

Synthèse : trois contrôles sont implémentés et bloquants dans `ci.yml` (tests unitaires, scan des secrets, smoke test des images avant publication) ; les autres sont recommandés et seront ajoutés selon la feuille de route ci-dessous.

Notes et références

Feuille de route proposée

- **Palier 1 (immédiat)** : JaCoCo et seuils de couverture, tests unitaires manquants (2.1), correction de la boucle de relance et des écarts au contrat (201, 404, `limit`), protection de la branche `main`.
- **Palier 2** : job `integration` (Testcontainers), CodeQL, Dependabot, Trivy avant le push.
- **Palier 3 (avec la pré-production)** : k6, Lighthouse CI, OWASP ZAP, E2E Playwright.

Anomalies relevées pendant l'analyse

- Front : `catchError` relance la requête en boucle en cas d'erreur HTTP (`olympic.service.ts`).
- API : les `POST` renvoient 200 au lieu du 201 défini dans `openapi.yaml`.
- API : un atelier inexistant renvoie 200 avec un corps vide au lieu d'une erreur 404.
- API : le paramètre `limit` de `GET /workshops` est ignoré.
- API : créer une notion déjà existante écrase la précédente sans erreur.
- Front : le lien « Go back » de la page 404 est inactif (`RouterLink` non importé dans `NotFoundComponent`).
- Tests : `JavaBasicAppApplicationTests` ne contient aucune méthode ; le chargement du contexte n'est jamais testé.
- Plateforme : Spring Boot 3.2.4 hors maintenance ; conteneurs exécutés en root ; `show-sql` actif.

Méthode de mesure

Couvertures mesurées le 15/10/2026 sur le commit `28b9346` (branche `ci/docker-build`) : `npx ng test --watch=false --code-coverage` pour le front ; `./gradlew clean test` avec JaCoCo injecté par un script d'initialisation Gradle temporaire, sans modifier le projet, code généré exclu, pour le back.

Fichiers du dépôt

- `.github/workflows/ci.yml` : pipeline (détection des applications modifiées, tests, scan des secrets, build, smoke test et push GHCR).
- `run-tests.sh` : exécution des tests des deux applications et collecte des rapports JUnit.
- `olympic-games-starter/karma.conf.js` : configuration Karma (Chrome Headless, rapports JUnit et couverture).
- `workshop-organizer/src/main/resources/specs/openapi.yaml` : contrat de l'API, référence des tests d'intégration et de sécurité.

Références

- Angular, guide des tests : https://angular.dev/guide/testing
- Spring Boot, tests : https://docs.spring.io/spring-boot/reference/testing/
- Testcontainers, module PostgreSQL : https://java.testcontainers.org/modules/databases/postgres/
- Grafana k6, seuils : https://grafana.com/docs/k6/latest/using-k6/thresholds/
- web.dev, Core Web Vitals : https://web.dev/articles/vitals
- Google Testing Blog, Code Coverage Best Practices (2020) : https://testing.googleblog.com/2020/08/code-coverage-best-practices.html
- OWASP Top 10 (2021) : https://owasp.org/Top10/ ; OWASP API Security Top 10 (2023) : https://owasp.org/API-Security/
- Gitleaks : https://github.com/gitleaks/gitleaks ; Trivy : https://trivy.dev ; OWASP ZAP : https://www.zaproxy.org
- Spring Boot, calendrier de support : https://spring.io/projects/spring-boot#support